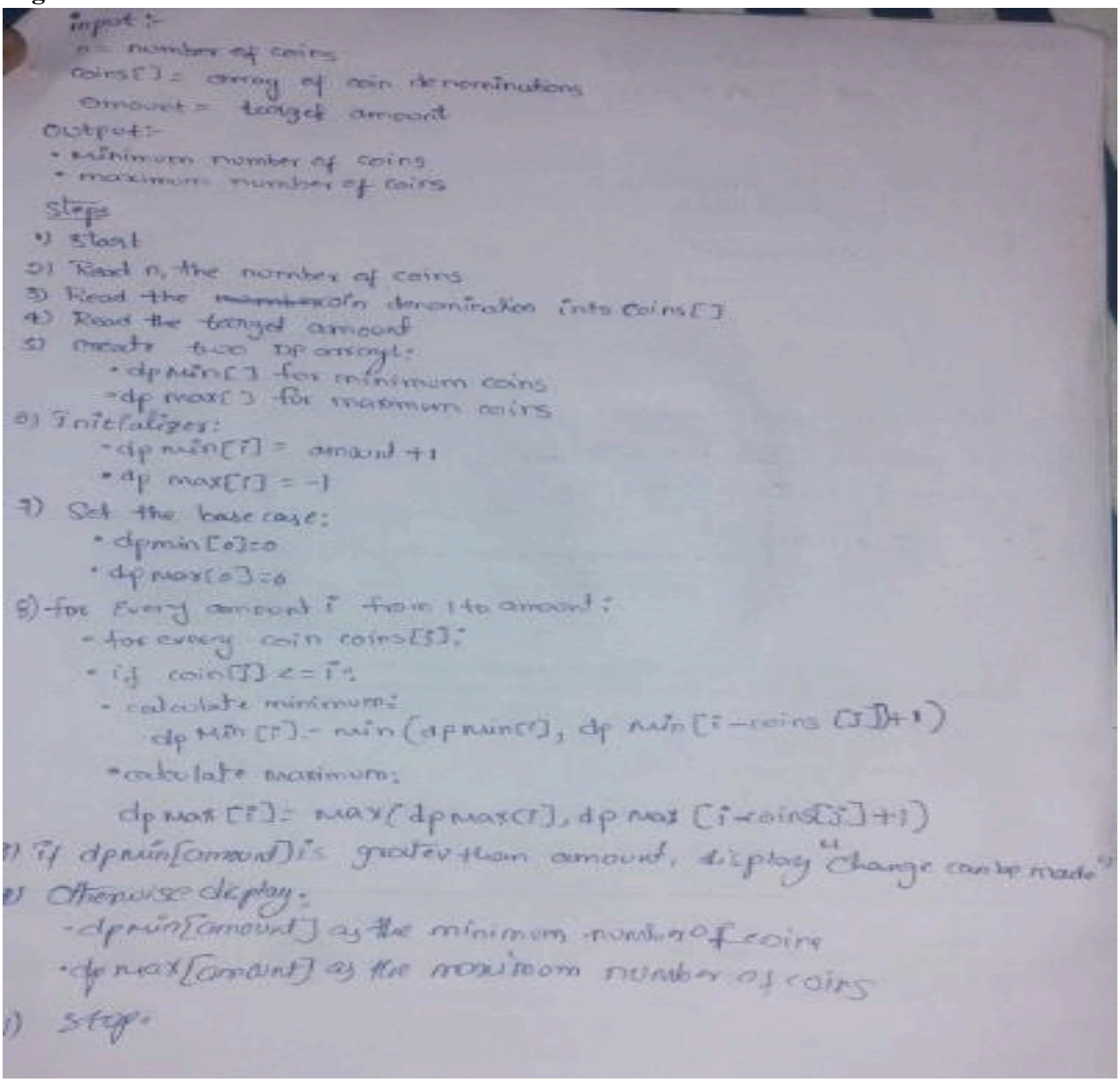
Practical: 7

Aim: Implementation of making a change problem using dynamic programming.

Algorithm:



PROGRAM

```
#include   <iostream>
#include  <algorithm>
using namespace std;

class CoinChange {
private:
    int coins[10];
    int n;
    int amount;

    int dpMin[100];
    int dpMax[100];

public:
    void getInput() {
        cout << "Enter number of coins: ";
        cin >> n;

        cout << "Enter coin denominations: ";
        for (int i = 0; i < n; i++) {
            cin >> coins[i];
        }

        cout << "Enter amount: ";
        cin >> amount;
    }

    void solve() {

        // Initialize DP arrays
        for (int i = 0; i <= amount; i++) {
            dpMin[i] = amount + 1;
            dpMax[i] = -1;
        }

        dpMin[0] = 0;
        dpMax[0] = 0;

        // Dynamic Programming
        for (int i = 1; i <= amount; i++) {
```

```
        for(intj=0;j<n;j++) {

            if (coins[j] <= i) {

                //Minimumnumber of coins
                dpMin[i]=min(dpMin[i],
                        dpMin[i - coins[j]] + 1);

                //Maximumnumber  of  coins
                if(dpMax[i-coins[j]]  !=  -1)  {
                dpMax[i]=max(dpMax[i],
                          dpMax[i - coins[j]] + 1);
                }
            }
        }
    }

    // Display result
    if(dpMin[amount]>amount) {
        cout<<"Changecannot be made." << endl;
    }
    else {
        cout<<"Minimumnumber of coins = "
            <<dpMin[amount] << endl;

        cout<<"Maximumnumber of coins = "
            <<dpMax[amount] << endl;
    }
  }
};

int main() {

  CoinChange c;

  c.get Input();
  c.solve();

  return 0;
}
```

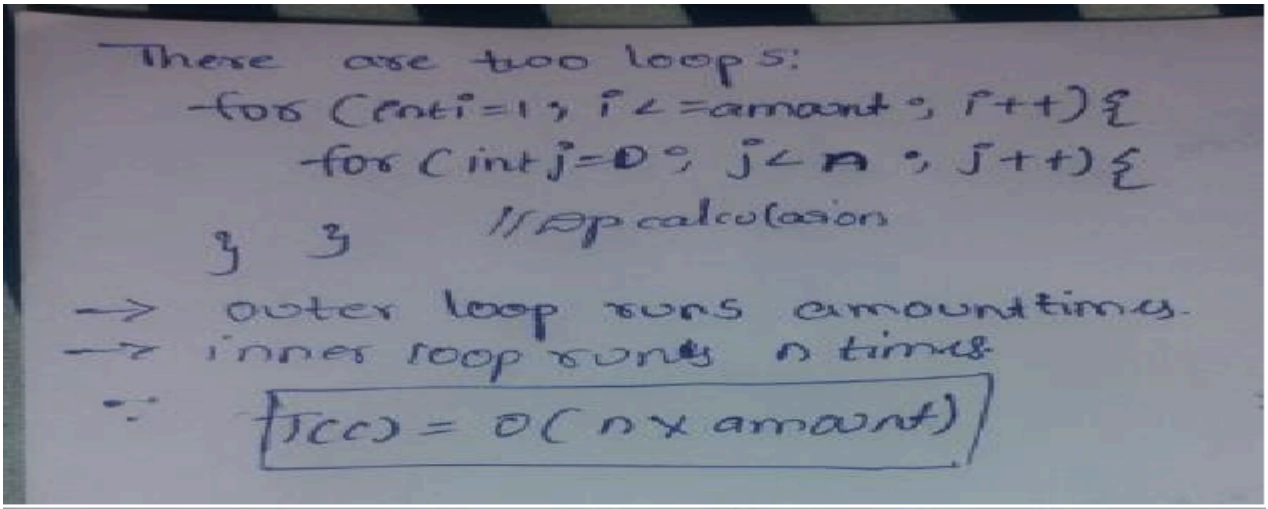
OUTPUT:

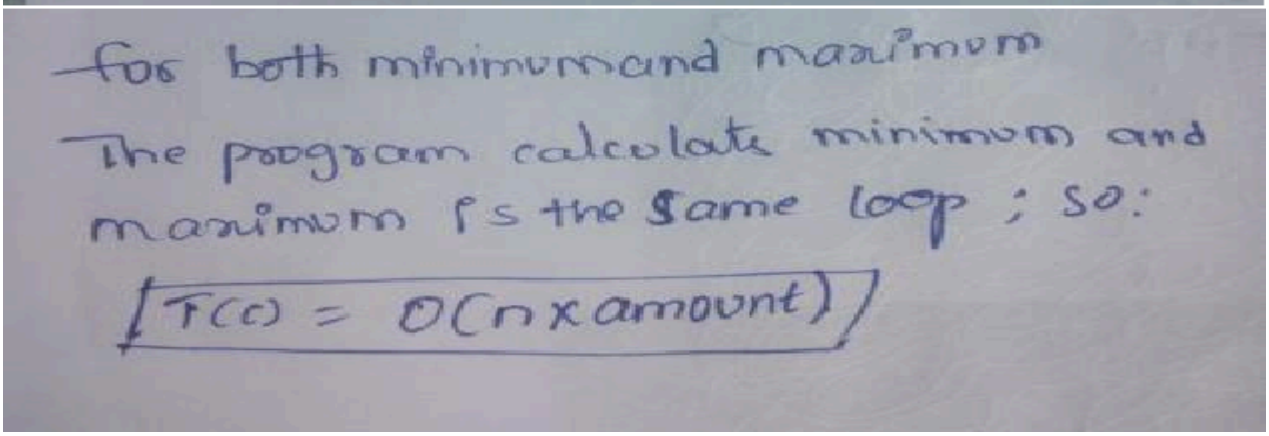
```
Output

Enter number of coins: 4
Enter coin denominations: 1
5
6
9
Enter amount: 10
Minimum number of coins = 2
Maximum number of coins = 10


=== Code Execution Successful ===
```

TIME COMPLEXITY:





CONCLUSSION:

Thus, the Making Change Problem was successfully implemented using Dynamic Programming in C++. The program finds the minimum and maximum number of coins required to make the given amount efficiently. Dynamic Programming avoids repeated calculations by storing previously computed results, improving the overall efficiency of the solution**.**